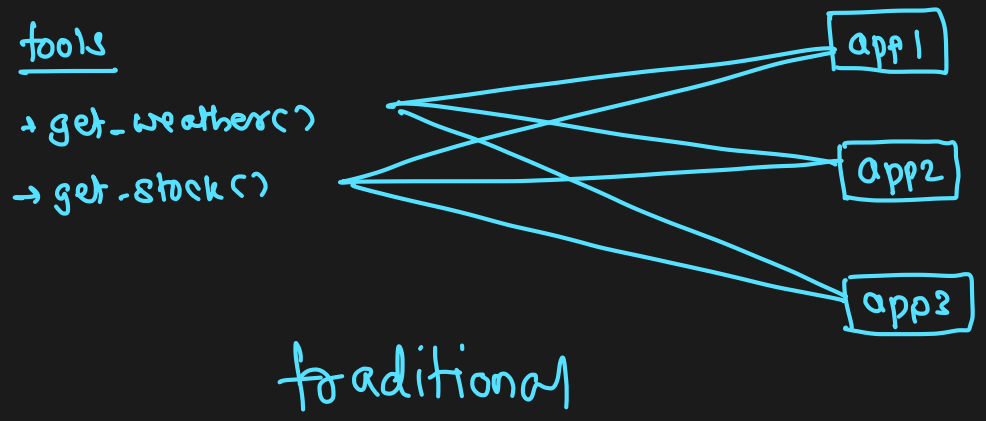
MCP

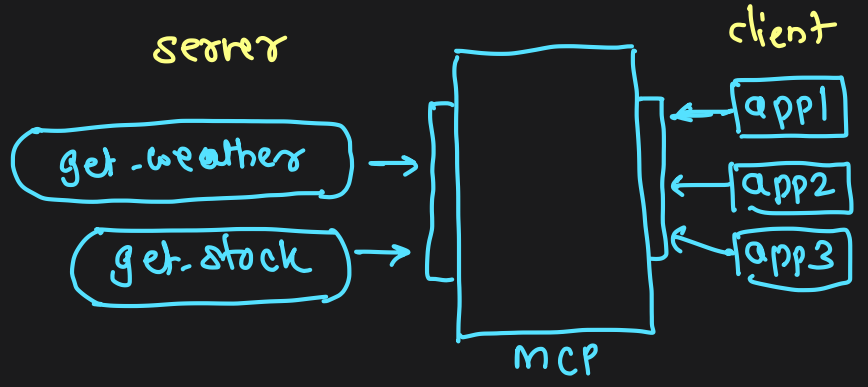
model context protocol

MCP

prompts & resources

- MCP is an open standard that enables AI applications to connect seamlessly with external data sources and tools
- It is often described as a **"USB-C port for AI applications"**—a universal connector that eliminates the need for custom integrations for every tool or data source
- Introduced by Anthropic in November 2024
- In December 2025, Anthropic donated MCP to the **Linux Foundation**, with co-founders including Anthropic, Block, and OpenAI
- Platinum members include AWS, Google, Microsoft, and Cloudflar

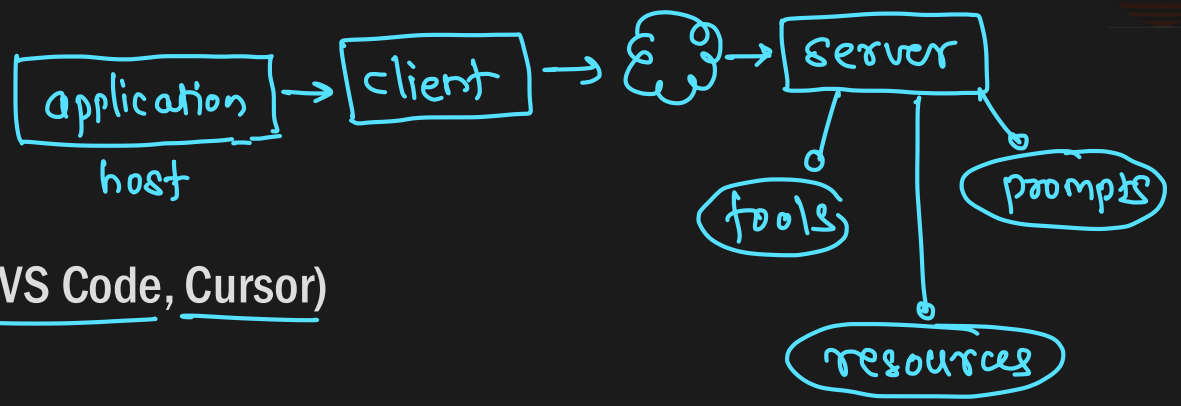




The "N x M" Problem

- Before MCP, connecting **N** AI models to **M** external tools required **N×M** custom integrations
- MCP reduces this to an **N+M** model
  - build one MCP server, and any compatible AI application can use it
- **Benefits**
  - Developers: Reduces development time; "integrate once" instead of rebuilding for every model
  - AI Applications: Access to an ecosystem of tools and data sources without custom code
  - End Users: More capable AI that can access personal data and take actions

Architecture

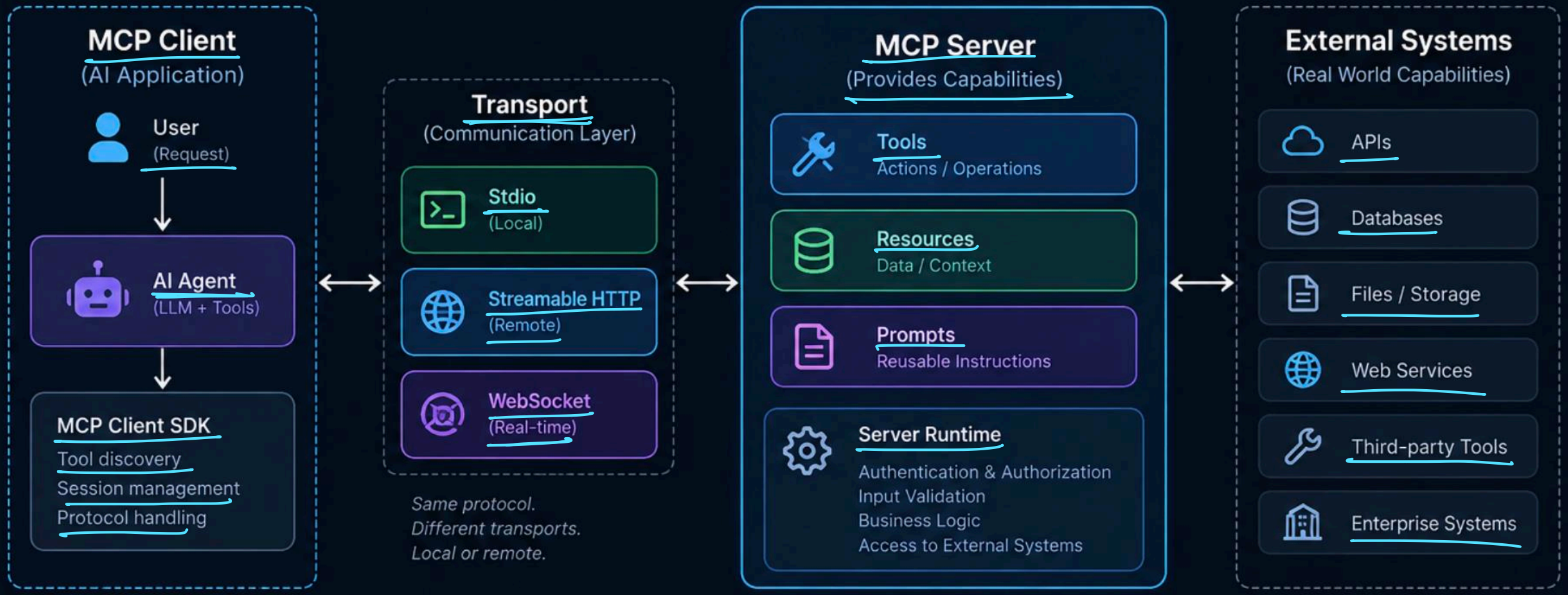


- MCP follows a client-host-server architecture
- **Host**
  - The **host** is the AI application itself (e.g., Claude Desktop, VS Code, Cursor)
  - It manages the overall session
  - Creates and manages multiple client instances
  - Enforces security policies and consent requirements
  - Coordinates AI/LLM integration and sampling
  - **Critical Security Boundary**
    - The host ensures servers cannot read the full conversation or "see into" other servers
- **Client**
  - A **client** is a connector inside the host
  - Each client maintains a **1:1 relationship** with exactly one server
  - Attaches protocol version and capabilities to every request
  - Routes messages bidirectionally
  - Maintains isolation between servers

Architecture

- **Server**
  - A **server** provides the actual capabilities (tools, data, prompts)
  - Servers are designed to be **extremely easy to build** and **highly composable**
  - Can be local processes (STDIO) or remote services (HTTP)
  - Focused on specific, well-defined responsibilities
  - Servers can offer three main types of features to clients
    - Resources
      - Context and data for the user or AI model to read
      - E.g. A database record, a file, or a document
    - Prompts
      - Templated messages and workflows
      - E.g. A pre-defined "summarize this article" workflow
    - Tools
      - Functions for the AI model to execute
      - E.g. get_weather, send_email, query_database

Architecture



Protocol Mechanics

- **Base Protocol: JSON-RPC 2.0**
  - All MCP messages use **JSON-RPC 2.0** formatting
  - Key message types
    - **Requests:** Include a unique ID; expect a response.
    - **Responses:** Successful results or errors.
    - **Notifications:** One-way messages with no ID and no expected response .
- **Stateful vs. Stateless Evolution**
  - **Early Spec (2025-03-26):** Stateful connections with persistent sessions
  - **Current Spec (2026-07-28):**
    - **Stateless, self-contained requests**
    - Every request carries its own protocol version and capabilities in the _meta field
  - **Why it matters:** Statelessness enables trivial horizontal scaling, load balancing, and failover for enterprise deployment
- **Capability Negotiation**
  - Clients and servers declare supported features during initialization
  - **Servers declare:** Tool support, resource subscriptions, prompt templates
  - **Clients declare:** Sampling support, elicitation handling, roots
  - **Extensions:** Optional features (e.g., Tasks for async operations, MCP Apps for interactive UI) are opt-in and negotiated

Security Framework



- **User Consent and Control**
  - Users must explicitly consent to and understand all data access and operations
- **Data Privacy**
  - Hosts must obtain consent before exposing user data to servers
- **Tool Safety**
  - Tools represent arbitrary code execution
  - Tool descriptions should be considered **untrusted** unless from a trusted server
- **Sampling Controls**
  - Users must approve LLM sampling requests, including the prompt and what results the server can see

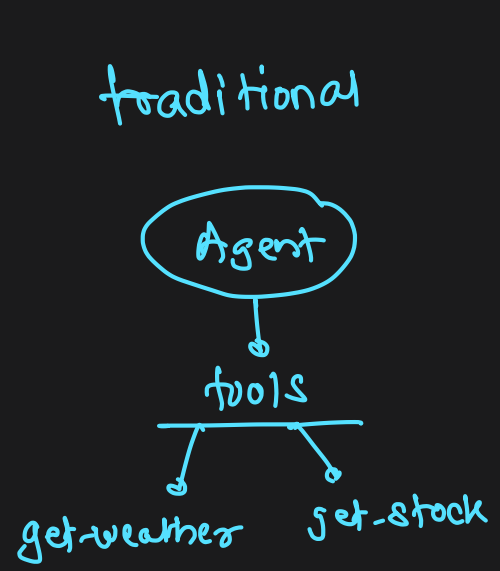
Benfits

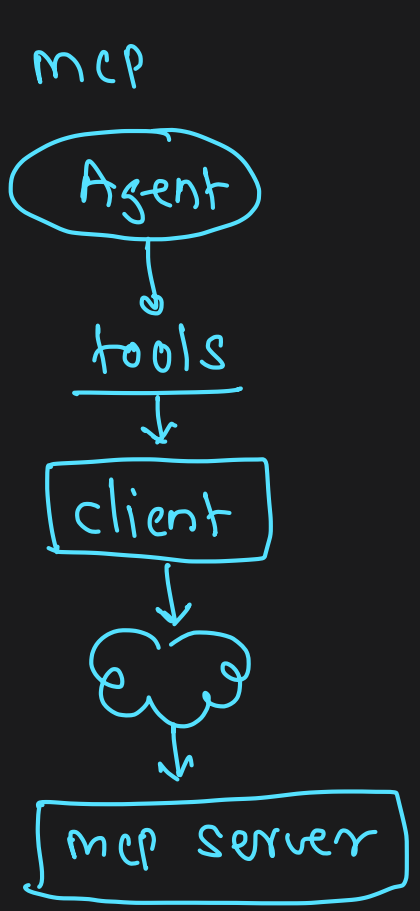


- **Standardized integration**
  - The biggest advantage is that MCP provides a **standard protocol** for connecting AI applications with external capabilities
- **Reusable integrations**
  - An MCP server can potentially be used by multiple MCP-compatible applications
- **Tool discovery**
  - MCP allows clients to discover capabilities dynamically
- **Separation of concerns**
  - MCP creates a clean architectural boundary
- **Supports different capabilities**
  - MCP isn't limited to tools, it can also be used to share dynamic prompts as well as resources
- **Framework independence**
  - An MCP server doesn't inherently need to be built with LangChain
- **Local and remote deployment**
  - MCP supports different transport mechanisms

Disadvantages

- Additional architectural complexity
- Another protocol to understand
- Additional latency
- Security becomes important
- Tool overload
- Debugging can become harder
- Not every application needs MCP
- **Version compatibility can be a challenge**





When to use and when not to use



- When to use MCP
  - Company-wide AI platform
  - Shared database access
  - Shared GitHub integration
  - Shared enterprise systems
  - Multiple AI agents
  - Multiple AI clients
  - Remote AI services
  - Reusable business capabilities
- When not to use MCP
  - Simple chatbot
  - One application
  - One or two internal functions
  - No external integrations
  - No need to reuse the integration

Implementation

Server

```python
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("Demo MCP Server")


@mcp.tool()
def add_numbers(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

@mcp.resource(
    "knowledge://python",
    mime_type="text/plain"
)
def python_knowledge() -> str:
    """Basic information about Python."""

    return """
Python is a high-level programming language.

Python is commonly used for:
- Artificial Intelligence
- Machine Learning
- Data Science
"""

@mcp.prompt()
def explain_topic(topic: str) -> str:
    """Generate a prompt for explaining a technical topic."""

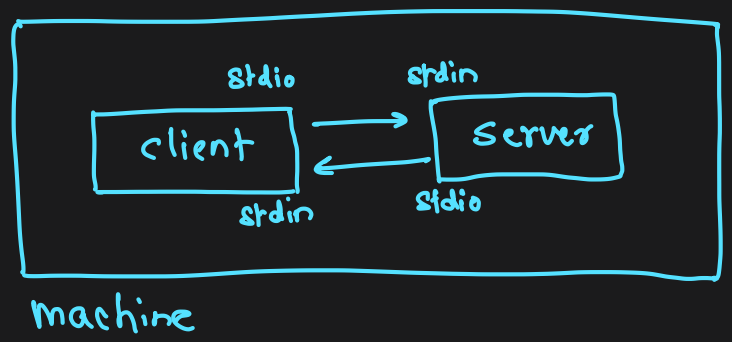
    return f"""
Explain {topic} in simple terms.

Include:
1. Definition
2. How it works
"""


if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8000)
```

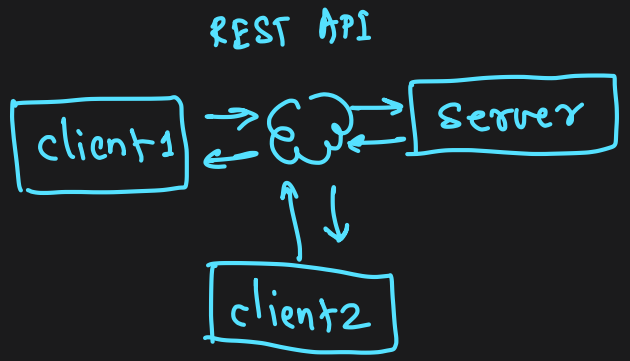
Transport

- In MCP, a **transport** defines **how the MCP client and MCP server communicate**
- The MCP protocol defines the messages exchanged; the **transport determines how those messages physically move between client and server**
- **There are 3 options: stdio, streamable http and SSE**
- **Stdio** → (Inter Process Communication)
  - This transport tunnels MCP messages over the standard input and output streams of a subprocess
  - The client launches the server as a local subprocess. Messages are exchanged via the server's stdin and stdout
  - **When to use it**
    - **Client and server are on the same machine**
    - **You are developing/testing an MCP server**
    - **An AI desktop application launches local MCP servers**
    - **You don't need a network server**



Transport: Streamable HTTP

- This is the recommended transport for remote, networked deployments
- It exposes the MCP server over a single HTTP endpoint (like http://localhost:8000/mcp) that supports both POST and GET requests
- The server runs as an independent process
- Clients send requests via HTTP POST
- The server can respond either with a single JSON object or by opening a Server-Sent Events (SSE) stream for streaming multiple messages back
- **When to use it**
  - MCP server is remote
  - Multiple clients need to access the server
  - Server is deployed in cloud
  - You need network-based access
  - You're building a production MCP service



Transport: SSE



- You might see references to an sse transport in older tutorials
- This was the **original HTTP transport** but has since been **deprecated**
- It was replaced by Streamable HTTP in a 2025 protocol revision
- It exists only for backward compatibility with clients that haven't migrated yet
- **Recommendation**
  - **Do not use it** for any new projects. Always choose streamable-http for web-based deployments

Client



```python
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            for tool in tools.tools:
                print(f"{tool.name}: {tool.description}")

            result = await session.call_tool(
                "add_numbers",
                {"a": 10, "b": 20}
            )
            print(result)


if __name__ == "__main__":
    asyncio.run(main())
```